

- Q8: 모든 연산이 10% 미만 → 구체적 병목 없음

hgTd

A?

L

r ?5r EL

L

5

5

VTn t

A

중첩 루프(Nested Loop) 조인:

- 가장 간단하지만 느림 $O(n*m)$
- 작은 데이터셋에는 괜찮음

해시 조인(Hash Join):

- 메모리가 있으면 매우 효율적 $O(n+m)$
- 메모리 부족하면 극적으로 느려짐

정렬 병합(Sort-Merge):

- 이미 정렬된 입력에 효율적
- 정렬 비용이 크면 비효율적

r D L

C

?:

VTn t

A

작은 그룹 수:

- 해시 집계 매우 효율적
- GROUP BY 병목 아님

큰 그룹 수:

- 메모리 오버플로우 위험
- 정렬 기반 집계 필요
- 디스크 I/O 증가

r ?

L

1

2

L

A

메모리 내 정렬:

- 매우 빠름

외부 정렬(External Sort):

- 메모리 초과 시 발생
- 다중 pass로 인한 디스크 I/O
- 성능 급격히 저하

r B5r ?:

L

psVYs T가

hgTe

B?

인덱스 없음:

- Q6 실행시간: 1000ms
- 전체 LINEITEM 테이블 스캔 필요

인덱스 있음 (L_SHIPDATE):

- Q6 실행시간: 50ms

- 20배 성능 개선

L

1ot Ysu9/YmYuY9vqVSuY 2

B7A

날짜별 파티셔닝:

- LINEITEM을 연도별로 분할
- 범위 검색 쿼리에서 스캔량 감소
- Q1 같은 쿼리에서 성능 향상

계약:

- 복잡한 조인 쿼리에서는 이득 적을 수 있음
- 파티션 프루닝(pruning) 기능 필요

B7B

1s가능t분날개

테이블:

```
| ID | Name | Price | Date |
|1|Apple|$1|2020|
|2|Banana|$0.5|2020|
```

1U가평균그t분날개

Price 컬럼만 필요하면:

Price: [1, 0.5, ...]

Date: [2020, 2020, ...]

필요한 컬럼만 로드

Lr ? ?:

hg7f

C7? VTnt t 는년

상용 DBMS (Oracle, SQL Server):

- 복잡한 조인 최적화 우수
- 벡터화 실행 등 최신 기법 적용
- Q17 같은 복잡한 쿼리에서 강함

오픈소스 (PostgreSQL):

- 간단한 쿼리에서는 경쟁력 있음
- 복잡한 쿼리 최적화는 뒤떨어짐
- 하지만 지속적으로 개선 중

C7A

1GB 스케일:

- 모든 데이터가 메모리에 올라옴
- 인덱싱, 파티셔닝의 효과 적음
- 대부분의 시스템 비슷한 성능

100GB 스케일:

- 메모리 관리가 중요해짐
- 시스템 간 성능 차이 증대
- 조인 순서 최적화의 중요성 증가

1TB 이상:

- 디스크 I/O가 절대적 병목
- 인덱싱, 파티셔닝 매우 중요
- 시스템 간 성능 격차 극대화

hg7g 별목기관

D7?

현대 데이터 특성:

- 비정규화된 스키마 (NoSQL의 영향)
- 복잡한 데이터 타입 (JSON, 배열, 맵)

- 다양한 정렬도(cardinality) 분포

TPC-H의 단순함:

- 정규화된 스키마만 사용
- 기본 데이터 타입만
- 비교적 균등한 데이터 분포

DA

TPC-H의 쿼리들:

- 대부분 10개 이하의 조인
- 명확한 WHERE 조건

현대 분석 워크로드:

- 20개 이상의 조인 가능 (DataLake)
- 중첩 쿼리, CTEs의 복잡한 사용
- 동적 쿼리 생성

DB

TPC-H의 한계:

- 정확한 매칭만 가능
- 유사도 기반 검색 없음
- 의미론적 분석 불가능

현대 요구사항:

- 고객 리뷰와 텍스트 유사도 검색
- 이미지 유사도 기반 추천
- 의미론적 검색과 정확한 검색의 결합

hgn

Y나는날 uqU6h

L

uqU6h

L

L

L

L

L

r ? 1

2

선

Q

r D 1

2

선

Q

i9p5

?7

L

1

2

Q

Q

A7

L

5

5

Q

B7

L

Q

5

Q

C7

LV는날 uqU6h

Q

99p

Q

D7

L

1

5

5

2

5

Q

Q

E7

는날

LuqU6h

5

7

Q